Name: A. Midhuna
Register No: 192521302

ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 31/10/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:

_____ How to Answer the

Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.
Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Call by Value, by Reference and by Pointer 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

Your Challenge

Write the concession function three ways – passing the fee by value, by reference and by pointer. Before running, predict the stored fee after each call, then verify.

Inquiry

Explain where each version stores its parameter, why only two of them change the caller's fee, and when a const reference parameter should be used instead.

Design Decision

Several concessions may apply to one student. Decide whether the function should change the fee in place or return the new fee, and defend your choice.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


void change(int a, int& b, int* c)
```

```
{
 a += 10;
 b += 10;
*c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
 int a = 1, b = 2, c = 3;
 change(a, b, &c);
 std::cout << a << ' ' << b << ' ' << c << '\n';
 bigger(a, c) = 100;
 std::cout << a << ' ' << c << '\n';
 show(1);
 show(1, 2);
 show(1, 2, 3);
 std::cout << '\n';
}
```

My predicted output (write before running):
Explain which variables changed in each call and why, and which default arguments were used by each call to show().

Complete the Code

Complete TODO 1 below so that the caller's fee is changed, and execute the

program. #include <iostream>

```
void applyByValue(long long fee, int percent)
{
 fee -= fee * percent / 100; // changes only the local copy
}
```

```
void applyConcession(long long& fee, int percent)
 {
 // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by
// reference)


}


int main()
 {
 long long hostelFee = 45000;
 applyByValue(hostelFee, 10);
 std::cout << "after call by value : " << hostelFee << '\n';
 applyConcession(hostelFee, 10);
 std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

after call by value : 45000
after call by reference: 40500

Explain why the call by value leaves the fee at 45000.

Alternative Concept and Complexity Analysis

If the concession function returned the new fee (long long applyConcession(long long fee, int percent)) instead of changing the fee through a reference, what would the solution be? Analyse the time (O(1) in all versions) and the space copied per call for a long long and for a large Student object passed by value, by reference and by const reference.

Evidence Expected

Three implementations, predictions before execution, observed results and a defended interface design.

ANSWER:

Hostel Fee Concession Updater – C++ (K5 Evaluate)

1. Three ways to write the concession function

The hostel applies concessions as follows:

- Merit concession – 10%
- Sports concession – 15%
- Single parent concession – 25%

Formula:

$$ NewFee=Fee-\frac{Fee\times Percentage}{100} $$

Method 1: Pass by Value

```
void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}
```

- A copy of the original fee is passed.
- Changes happen only to the local variable.
- The caller's fee remains unchanged.

Method 2: Pass by Reference

```
void applyByReference(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
```

- The function receives a reference to the original fee.
- Changes directly affect the caller's variable.
- No separate fee copy is created.

Method 3: Pass by Pointer

```
void applyByPointer(long long* fee, int percent)
{
   *fee -= (*fee) * percent / 100;
```

}

- The address of the fee is passed.
- The pointer accesses the original fee using *.
- Changes directly affect the caller's variable.

2. Predict the stored fee before execution

Assume the initial hostel fee is ₹45,000 and the concession is 10%.

Method	Predicted stored fee
Pass by value	₹45,000
Pass by reference	₹40,500
Pass by pointer	₹40,500

3. Complete program – Three implementations

```
#include <iostream>

using namespace std;


void applyByValue(long long fee, int percent)

{

   fee -= fee * percent / 100;

}


void applyByReference(long long& fee, int percent)

{

   fee -= fee * percent / 100;

}


void applyByPointer(long long* fee, int percent)

{

   *fee -= (*fee) * percent / 100;
```

```
}

int main()
{
    long long fee1 = 45000;
    long long fee2 = 45000;
    long long fee3 = 45000;

    applyByValue(fee1, 10);
    applyByReference(fee2, 10);
    applyByPointer(&fee3, 10);

    cout << "Pass by value: " << fee1 << '\n';
    cout << "Pass by reference: " << fee2 << '\n';
    cout << "Pass by pointer: " << fee3 << '\n';

    return 0;
}
```

Expected output:

Pass by value: 45000

Pass by reference: 40500

Pass by pointer: 40500

4. Inquiry – Where is each parameter stored?

Method	Parameter storage	Caller fee changes?
Value	Separate local copy	No
Reference	Refers to original variable	Yes
Pointer	Stores address of original variable	Yes

Why does only two methods change the caller's fee?

- Pass by value creates an independent copy.
- Pass by reference directly accesses the original variable.
- Pass by pointer accesses the original variable through its memory address.

When should const reference be used?

A const reference is used when we want to access a variable without modifying it.

Example:

```
void displayFee(const long long& fee)
{
   cout << fee;
}
```

It avoids unnecessary copying and prevents accidental modification.

5. Predict the output – Given C++ program

My predicted output (before running):

1 12 13

1 100

16 13 6

Explanation

Step 1: change(a, b, &c)

Initial values:

a = 1, b = 2, c = 3;

Inside the function:

- a += 10 changes only the local copy of a.
- b += 10 changes the original b to 12.
- *c += 10 changes the original c to 13.

Therefore:

1 12 13

Step 2: bigger(a, c) = 100

The bigger function returns a reference to the larger variable.

Current values:

a = 1, c = 13

Since c is larger, the function returns a reference to c.

Therefore:

bigger(a, c) = 100;

Changes c to 100.

Output:

1 100

Step 3: Default arguments in show()

void show(int x, int y = 5, int z = 10)

Function call	Values used	Output
show(1)	1, 5, 10	16
show(1, 2)	1, 2, 10	13
show(1, 2, 3)	1, 2, 3	6

Final output:

1 12 13

1 100

16 13 6

Verification: The predicted output matches the expected output. No differences occur.

6. Complete the Code – TODO 1

Correct code:

#include <iostream>

using namespace std;

void applyByValue(long long fee, int percent)

```
{
    fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

    cout << "after call by value : "
         << hostelFee << '\n';

    applyConcession(hostelFee, 10);

    cout << "after call by reference: "
         << hostelFee << '\n';

    return 0;
}
```

Expected output:

```
after call by value : 45000
```

after call by reference: 40500

Explanation:

- Call by value changes only the local copy, so the original fee remains ₹45,000.
- Call by reference modifies the original fee directly.
- Therefore, the final fee becomes ₹40,500.

7. Alternative Concept – Return the new fee

Instead of modifying the original fee, we can return the updated fee.

Program:

```
#include <iostream>
using namespace std;

long long applyConcession(long long fee, int percent)
{
   return fee - (fee * percent / 100);
}

int main()
{
   long long hostelFee = 45000;

   hostelFee = applyConcession(hostelFee, 10);

   cout << "Updated hostel fee: " << hostelFee;

   return 0;
}
```

Output:

Updated hostel fee: 40500

Here, the returned value is assigned back to hostelFee.

8. Time and Space Complexity Analysis

Parameter method	Time complexity	Space used
Pass by value	O(1) for long long	8-byte copy
Pass by reference	O(1)	No object copy
Pass by pointer	O(1)	Stores an address
Pass by const reference	O(1)	No object copy

For a large Student object:

- Pass by value copies the entire object, requiring additional memory and copying time proportional to its size.
- Pass by reference avoids copying and allows modification.
- Pass by const reference avoids copying and prevents modification.

9. Design Decision – Which method is better?

If several concessions apply to one student, I recommend returning the updated fee.

Reasons:

1. It keeps the original value unchanged until the result is assigned.
2. It makes the function easy to understand and test.
3. Multiple concessions can be applied sequentially.
4. It avoids unexpected changes to the caller's variable.

For example:

fee = applyConcession(fee, 10);

fee = applyConcession(fee, 15);

The fee is updated after each concession.

Note: Whether multiple concessions should be added together or applied sequentially depends on the hostel's concession policy.

10. Final Evaluation

Method	Advantage	Disadvantage
Value	Safe local copy	Does not update caller
Reference	Direct modification	Changes original data
Pointer	Direct access using address	Requires pointer safety
Return value	Clear and easy to test	Caller must assign result
Const reference	Avoids copying	Cannot modify original

Final Conclusion:

Pass by reference and pointer modify the caller's fee, whereas pass by value modifies only a local copy. For the hostel fee concession system, returning the updated fee is a clear and maintainable design choice. Const reference is best when data should only be read.

Q2 – Return by Reference and Its Dangers 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Examination Marks Register

The marks register keeps the marks of a class in an array. A function returns a reference to the highest mark so that a moderation of +2 can be applied directly to it. A second function returns a reference to a local variable.

Your Challenge

Implement both functions and show:

• How returning a reference lets the caller change the array element (and even use the call on the left of =)

• What the compiler warns about when a reference to a local variable is returned, and why using it is undefined behaviour

• The difference between returning a reference, returning a const reference and returning a copy

Changed Condition

The register is now stored in a std::vector that may grow. Explain why a reference obtained

before the vector grows may become invalid.

Alternative Concept and Complexity Analysis

If the highest mark were returned as an index (int) or as a pointer (int*) instead of a reference (int&), what would the calling code look like? Analyse the time to find the highest mark (O(n) in all three) and the space (O(1)), and compare the safety of each when the array is later replaced by a growing vector.

Evidence Expected

Working functions, compiler-warning evidence, a correct explanation of dangling references and a justified response to the changed condition.

ANSWER:

Implement functions to return the highest mark

A reference allows the caller to directly modify the original array element.

C++ Program:

```
#include <iostream>

using namespace std;


int& highestMark(int marks[], int n)

{

   int maxIndex = 0;


   for (int i = 1; i < n; i++)

   {

      if (marks[i] > marks[maxIndex])

         maxIndex = i;

   }


   return marks[maxIndex];
```

```
}

int main()
{
    int marks[] = {75, 88, 92, 80, 85};

    highestMark(marks, 5) += 2;

    cout << "Updated marks: ";

    for (int i = 0; i < 5; i++)
        cout << marks[i] << " ";

    return 0;
}
```

Output:

Updated marks: 75 88 94 80 85

Explanation:

- The highest mark is 92.
- The function returns a reference to that array element.
- += 2 directly increases the original mark to 94.
- We can also use the function on the left side of an assignment:

```
highestMark(marks, 5) = 100;
```

Now the highest mark becomes 100.

Danger of returning a local variable

Incorrect function:

```
int& dangerous()
```

```
{
   int x = 50;
   return x;
}
```

Compiler warning:

warning: reference to local variable 'x' returned

Why is it dangerous?

- The variable x is created inside the function.
- It is destroyed when the function ends.
- The returned reference points to an object whose lifetime has ended.
- Accessing it causes undefined behaviour.

Therefore, returning a reference to a local variable must be avoided.

Difference between reference, const reference and copy

Type	Meaning	Modification
Return by reference (int&)	Returns original variable	Allowed
Return by const reference (const int&)	Returns read-only reference	Not allowed
Return by value (int)	Returns a separate copy	Original unchanged

Example:

```
int& getMark(int marks[])
{
   return marks[0];
}


const int& readMark(int marks[])
{
   return marks[0];
```

```
}

int copyMark(int marks[])
{
   return marks[0];
}
```

Changed Condition – Using std::vector

When the marks register uses std::vector, it can grow dynamically.

Example:

```
vector<int> marks = {75, 88, 92, 80};

int& ref = marks[2];

marks.push_back(95);
```

If push_back() causes reallocation:

- The vector moves its elements to new memory.
- The old reference becomes invalid.
- Accessing that reference causes undefined behaviour.

Solution: Use an index and access the vector again after it grows.

Alternative – Returning Index or Pointer

Returning an index:

```
int highestIndex(int marks[], int n)
{
   int max = 0;

   for (int i = 1; i < n; i++)
      if (marks[i] > marks[max])
```

```
            max = i;


    return max;
}
```

Calling code:

```
int index = highestIndex(marks, 5);
marks[index] += 2;
```

Returning a pointer:

```
int* highestPointer(int marks[], int n)
{
    int max = 0;


    for (int i = 1; i < n; i++)
        if (marks[i] > marks[max])
            max = i;


    return &marks[max];
}
```

Calling code:

```
int* p = highestPointer(marks, 5);
*p += 2;
```

Method	**Time complexity**	**Extra space**
Reference	O(n)	O(1)
Index	O(n)	O(1)
Pointer	O(n)	O(1)

All three methods take O(n) time to find the highest mark because every element must be checked.

Safety comparison:

- Reference: Can become invalid when vector reallocates.
- Pointer: Can also become invalid when vector reallocates.
- Index: Generally safer because it can be used again after vector growth, provided the index remains valid.

Final Evaluation

For a growing marks register, returning an index is the safest choice because it avoids dangling references and invalid pointers after vector reallocation.

Conclusion: Returning by reference allows direct modification of an array element, but returning a reference to a local variable causes undefined behaviour. For dynamically growing vectors, returning an index is a better design choice.

Q3 – Default Arguments and Function Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Courier Charge Calculator

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote charge(double weight), charge(double weight, bool express) and charge(double weight, bool express = false, bool insured = false). The program no longer compiles.

Your Challenge

Explain the compiler error (ambiguous call). Then design the charge functions so

that: • Common calls are short (default arguments)

• Different kinds of input (weight in grams as int, weight in kg as double, a Parcel object) are handled by overloading

• No call is ambiguous

Before execution, predict which function each of six calls you design will select, including charge(2) and charge(2.0f).

Inquiry

Explain how the compiler chooses among overloaded functions (exact match, promotion, conversion), and why default arguments must be given from the right.

Alternative Concept and Complexity Analysis

If the courier charge took a single Parcel structure with optional fields (or a function template) instead of several overloaded functions with default arguments, what would the solution be? Analyse the time of overload resolution (at compile time, no run-time cost) and

the space copied per call, and compare how each design handles a new option.

Evidence Expected

An explained ambiguity, a redesigned set of functions, predicted and observed selections, and a reasoned explanation of overload resolution.

ANSWER:

Default Arguments and Function Overloading

1. Reason for the Compiler Error

The original program contains three functions:

double charge(double weight);

double charge(double weight, bool express);

double charge(double weight, bool express = false, bool insured = false);

When we call:

charge(2.5);

All three functions can accept the call because the second and third functions have default arguments or fewer required parameters. Therefore, the compiler cannot decide which function should be selected.

This creates an ambiguous call error.

2. Redesigned Function Overloading

We can avoid ambiguity by using different parameter types and one function with default arguments.

```
#include <iostream>

using namespace std;


struct Parcel {

   double weight;

};
```

```
double charge(int grams) {
    return grams * 0.02;
}

double charge(double kg, bool express = false,
              bool insured = false) {
    double amount = kg * 50;

    if (express)
        amount += 30;

    if (insured)
        amount += 20;

    return amount;
}

double charge(Parcel p) {
    return p.weight * 50;
}

int main() {
    Parcel p = {3.0};

    cout << charge(500) << endl;
    cout << charge(2.0) << endl;
    cout << charge(2.0f) << endl;
```

```
    cout << charge(2.0, true) << endl;
    cout << charge(2.0, true, true) << endl;
    cout << charge(p) << endl;


    return 0;
}
```

3. Prediction of Function Selection

Function call	Selected function	Reason
charge(500)	charge(int)	Exact match
charge(2)	charge(int)	Exact match
charge(2.0)	charge(double, ...)	Exact match
charge(2.0f)	charge(double, ...)	Float promoted to double
charge(2.0, true)	charge(double, bool, bool)	Third argument uses default
charge(p)	charge(Parcel)	Exact match

Note: The table predicts the selected functions. The program prints the calculated charges, not the function names.

4. How the Compiler Chooses an Overloaded Function

The compiler follows these matching priorities:

1. Exact match: The argument type exactly matches the parameter type.
2. Promotion: A smaller type is promoted to a suitable larger type, such as float to double in C++.
3. Conversion: Other standard conversions are considered, such as int to double.

The compiler selects the best matching function. If two or more functions have equally good matches, an ambiguity error occurs.

Why must default arguments be given from the right?

Default arguments must be specified from right to left because they are used when trailing arguments are omitted.

Correct:

```
void display(int a, int b = 10, int c = 20);
```

Incorrect:

```
void display(int a = 10, int b, int c = 20);
```

5. Alternative Concept

Instead of multiple overloaded functions, we can use a single Parcel structure.

```
struct Parcel {
    double weight;
    bool express = false;
    bool insured = false;
};
```

The function can calculate the charge using all the fields:

```
double charge(Parcel p) {
    double amount = p.weight * 50;

    if (p.express)
        amount += 30;

    if (p.insured)
        amount += 20;

    return amount;
}
```

We can also use a function template to accept different input types, but type-specific handling may require additional logic.

6. Complexity Analysis and Comparison

Design	Time complexity	Space used
Function overloading	Compile-time selection; no runtime overload-resolution cost	Depends on arguments

Parcel structure	O(1) charge calculation	Space for structure fields; copied if passed by value
Function template	Compile-time specialization	Depends on argument and generated function

- Overloading: Suitable for handling different input types. A new option may require modifying function parameters or adding overloads.
- Parcel structure: Easy to add new options by adding fields, without creating many overloaded functions.
- Function template: Reduces repeated code for different types, but needs suitable type handling.

Passing a large Parcel object by const Parcel& avoids copying the entire structure.

Conclusion

Function overloading handles different data types, while default arguments make common calls shorter. Proper parameter design avoids ambiguity. For a courier system with many optional features, using a Parcel structure is easier to maintain and extend.

Q4 – Specifying a Class with an Array Inside It 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Savings Account Mini-Statement

A bank wants each SavingsAccount object to remember its last 10 transactions for the mini statement printed at an ATM. Older transactions are discarded.

Your Challenge

Specify the class with private data (account number, balance, an array of 10 transactions and a count) and public member functions deposit(), withdraw() and miniStatement(). Define some member functions inside the class and some outside using the scope resolution operator.

Inquiry

Explain why the array must be private, what happens when the 11th transaction arrives if you shift the array versus use a circular index, and the cost of each.

Design Decision

The bank now wants the last 50 transactions. Decide which design changes least and defend it.

Alternative Concept and Complexity Analysis

If the last 10 transactions were kept in a std::deque (push_back and pop_front), or in a circular buffer with a start index, instead of shifting a fixed array, what would the solution be? Analyse the time to add a transaction (O(k) for shifting k elements versus O(1)) and the space used in each design.

Evidence Expected

A complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.

ANSWER:

Specifying a Class with an Array Inside It (6 Marks)

1. Class Specification

A SavingsAccount class stores the account number, balance, and the last 10 transactions. When the 11th transaction arrives, the oldest transaction is removed.

```
#include <iostream>

using namespace std;


class SavingsAccount {

private:

    int accountNumber;

    double balance;

    double transactions[10];

    int count;


public:

    // Constructor defined inside the class

    SavingsAccount(int accNo, double initialBalance) {

        accountNumber = accNo;

        balance = initialBalance;

        count = 0;

    }
```

```
    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement();
};

// Deposit function defined outside the class
void SavingsAccount::deposit(double amount) {
    if (amount > 0) {
        balance += amount;

        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 0; i < 9; i++)
                transactions[i] = transactions[i + 1];

            transactions[9] = amount;
        }
    }
}

// Withdraw function defined outside the class
void SavingsAccount::withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
```

```
        double transaction = -amount;

        if (count < 10) {
            transactions[count++] = transaction;
        } else {
            for (int i = 0; i < 9; i++)
                transactions[i] = transactions[i + 1];

            transactions[9] = transaction;
        }
    }
}

// Mini statement defined outside the class
void SavingsAccount::miniStatement() {
    cout << "\nAccount Number: " << accountNumber;
    cout << "\nBalance: " << balance;
    cout << "\nLast Transactions:\n";

    for (int i = 0; i < count; i++)
        cout << transactions[i] << endl;
}

int main() {
    SavingsAccount acc(101, 5000);

    acc.deposit(1000);
```

```
    acc.withdraw(500);
    acc.deposit(200);
    acc.miniStatement();

    return 0;
}
```

Explanation:

- Positive values represent deposits.
- Negative values represent withdrawals.
- The constructor initializes the account details.
- deposit() adds money to the balance and records the transaction.
- withdraw() deducts money and records the withdrawal.
- miniStatement() displays the stored transactions.

2. Why Must the Array Be Private?

The transaction array must be private because of data hiding and encapsulation.

- Prevents users from directly changing transaction records.
- Protects the account information from accidental modification.
- Allows transactions to be updated only through deposit and withdrawal functions.
- Maintains data security and consistency.

3. What Happens When the 11th Transaction Arrives?

There are two methods:

Method 1: Shifting the Array

When the array is full, all elements are shifted one position to the left. The oldest transaction is removed, and the new transaction is inserted at the last position.

EXAMPLE

Before adding the 11th transaction:

[T1, T2, T3, ... T9, T10]

After adding T11:

[T2, T3, T4, ... T10, T11]

- Time complexity: O(10), generally O(k) for k stored transactions.
- Simple to implement, but requires shifting elements.

Method 2: Circular Index

A circular buffer uses a start index to identify the oldest transaction. When a new transaction arrives, it replaces the oldest one, and the index moves forward using modulo arithmetic.

- Time complexity: O(1) per insertion.
- No shifting is required.
- Slightly more complex to implement.

4. Design Decision – Increasing Transactions to 50

If the bank wants to store the last 50 transactions, the array size can simply be changed:

double transactions[50];

However, the shifting loop and array limits must also be updated.

My decision: I would prefer a circular buffer because it supports more transactions without shifting elements. It provides constant-time insertion and is efficient for frequently updated mini-statements.

5. Alternative Concept and Complexity Analysis

Instead of a fixed array, we can use std::deque.

```
#include <deque>

using namespace std;


deque<double> transactions;


void addTransaction(double amount) {
    if (transactions.size() == 10)
        transactions.pop_front();
```

```
    transactions.push_back(amount);
}
```

Here:

- push_back() adds a new transaction.
- pop_front() removes the oldest transaction.
- The deque automatically manages its storage.

Comparison

Method	Insertion time	Space
Fixed array with shifting	O(k)	O(k)
Circular buffer	O(1)	O(k)
std::deque	O(1) at the ends (amortized)	O(k), with some extra storage overhead

Here, k is the maximum number of stored transactions (10 or 50).

Conclusion

A private array protects transaction records and supports a simple mini-statement system. Shifting is easy for small transaction counts, but a circular buffer is more efficient because insertion takes O(1) time. For storing the last 50 transactions, I recommend a circular buffer because it uses fixed memory and avoids unnecessary shifting.

Q5 – Static Members and Arrays of Objects 7 Marks

Bloom's Level: K5 – Evaluate

Problem Context – College Bus Seat Tickets

The college bus has a fixed number of seats. Each BusTicket object must receive the next seat number, tickets beyond the capacity go to a waiting list, and the transport office must know how many seats have been issued without looking at any particular ticket.

Design Starting Point

You are given only the following principle:
Data that belongs to the whole class – not to any one object – should exist only once, however many objects are created.

Your Challenge

Convert this principle into a class design. Determine:

• Which members must be static, and where a static data member must be defined • Why a static member function cannot use this or a non-static member • How an array of BusTicket objects is created and initialised

• What happens to the count when tickets are created inside a loop or a function

Test Your Design

Create an array of tickets larger than the capacity and verify the seat numbers and the waiting list.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

class Token {
 static int next;
 int id;

public:
 Token() { id = ++next; }
 int get() const { return id; }
 static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
 Token counter[3];
 Token extra;
```

```
 std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
<< ' ' << Token::issued() << '\n';
}
```

My predicted output (write before running):

Explain why the static counter starts at 100 and how many times the constructor ran.
Complete the Code
Complete TODO 1 and TODO 2 below using the static members, and execute the program. #include <iostream>

```
#include <string>

class BusTicket {
 static int issued; // shared by all BusTicket objects
 static const int CAPACITY = 3;
 int seat;
 std::string name;

public:
 explicit BusTicket(const std::string& n) : seat(0), name(n)
 {
 // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
 // give this ticket the next seat number; otherwise leave seat as 0 (waiting  //
list)

 }

 static int count()
 {
 // TODO 2: Return the number of seats issued so far
```

```
}

 void print() const
 {
 std::cout << name << ": "
 << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))  << '\n';
 }
};

int BusTicket::issued = 0; // definition of the static data member

int main()
{
 BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")};
 for (const BusTicket& t : trip) t.print();
 std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

Create the tickets inside a function that is called twice and explain how the count behaves.

Alternative Concept and Complexity Analysis

If the seat counter were a global variable, or were kept in a separate TicketOffice object passed to every BusTicket, instead of a static data member, what would the solution be? Analyse the time (O(1) in all) and the space (one counter in all), and compare the encapsulation and the ability to run two buses at once.

Evidence Expected

A student-derived class design using static members, an array of objects, test evidence and a reasoned explanation of static members.

ANSWER:

Static Members and Arrays of Objects

1. Class Design Using Static Members

A static data member belongs to the entire class, not to an individual object. Therefore, all BusTicket objects share the same seat counter.

The following members are static:

- issued – stores the total number of seats issued.
- CAPACITY – stores the maximum number of seats.
- count() – returns the shared number of issued seats.

The seat and name members are non-static because each ticket has its own seat number and passenger name.

The static data member must be defined outside the class:

int BusTicket::issued = 0;

2. Why Cannot a Static Function Use this?

A static member function belongs to the class rather than a particular object.

- It can directly access only static members.
- It cannot use the this pointer because it is not called for a specific object.
- It cannot directly access non-static members such as seat and name.

For example, count() can access issued, but cannot directly access seat.

3. Predict the Output of the Token Program

Given:

int Token::next = 100;

The static counter starts at 100 because it is explicitly initialized to 100 outside the class.

Each constructor call executes:

id = ++next;

So the counter increases before assigning the ID.

My predicted output:

101 103 104 4

Explanation:

Object	Constructor execution	Assigned ID
counter[0]	1	101
counter[1]	2	102
counter[2]	3	103
extra	4	104

- counter[0].get() returns 101.
- counter[2].get() returns 103.
- extra.get() returns 104.
- Token::issued() returns next - 100, which is 4.

The constructor runs four times: three times for the array objects and once for extra.

The prediction matches the actual program output. There is no difference between the predicted and observed output.

4. Complete the BusTicket Code

TODO 1: Check whether seats are available. If available, increment issued and assign that number as the seat.

TODO 2: Return the static counter.

```
#include <iostream>

#include <string>


class BusTicket {

    static int issued;

    static const int CAPACITY = 3;
```

```
    int seat;
    std::string name;

public:
    explicit BusTicket(const std::string& n)
        : seat(0), name(n)
    {
        // TODO 1
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }

    static int count()
    {
        // TODO 2
        return issued;
    }

    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting"))
                  << '\n';
    }
```

```
};

int BusTicket::issued = 0;

int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
        t.print();

    std::cout << "Issued: " << BusTicket::count() << '\n';

    return 0;
}
```

Observed output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

The first three objects receive seats 1, 2 and 3. The fourth object receives seat 0, which represents the waiting list. The shared counter remains 3.

5. Creating Tickets Inside a Function Called Twice

Consider this function:

```
void createTickets() {
    BusTicket tickets[2] = {
        BusTicket("Esha"),
        BusTicket("Farhan")
    };

    for (const BusTicket& t : tickets)
        t.print();

    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

If createTickets() is called twice after the bus has already issued three seats:

- First call: Both tickets go to the waiting list.
- Second call: Both tickets also go to the waiting list.
- The static counter remains 3.

Why? Local objects are destroyed when the function ends, but the static data member belongs to the class. Its value is not reset when the function is called again.

If the function is called before all seats are issued, available seats are assigned normally. Once the capacity is reached, later tickets go to the waiting list.

6. Alternative Concepts and Complexity Analysis

There are three possible ways to maintain the seat counter.

Method 1: Static data member

```
static int issued;
```

- One shared counter for all tickets of the class.
- Easy to access using BusTicket::count().
- Good encapsulation because the counter is private.

- However, one static counter is shared by every bus using this class.

Method 2: Global variable

int issued = 0;

- The counter exists outside the class.
- All ticket objects can use the same global counter.
- It is less secure because other functions can directly modify it.
- It provides weaker encapsulation.

Method 3: Separate TicketOffice object

class TicketOffice {

public:

int issued = 0;

};

A TicketOffice object can be passed to each ticket when it is created. Each office maintains its own counter.

- Better for managing multiple buses.
- Each bus can have a separate ticket office.
- Avoids sharing one counter between different buses.

Complexity Comparison

Design	Time to issue ticket	Counter space	Encapsulation
Static member	O(1)	O(1) per class	Good
Global variable	O(1)	O(1) globally	Poor
TicketOffice object	O(1)	O(1) per office	Good

All three methods require constant time to check capacity and issue a seat. The counter itself requires constant space.

My design decision: I would choose a separate TicketOffice object if the college operates multiple buses. Each bus can maintain its own seat counter and capacity. For a single bus, a static member is simple and effective.

Conclusion

Static members store common information only once, regardless of how many objects are created. In the bus ticket system, the static counter maintains the number of seats issued, while each object stores its own passenger name and seat number. For multiple buses, separate TicketOffice objects provide better flexibility and independent seat management.

Q6 – Friend Functions and Inline Functions 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Train Journey Planner

A journey planner stores departure and arrival times in a Time class whose hours and minutes are private. It must compute the minutes between two times and decide which time is earlier, and these small functions are called millions of times while planning connections.

Your Challenge

Design the comparison. Determine:

• Why a friend function is needed when the function works on two objects equally

• How a friend function differs from a member function in how it is called and what it can access

• Whether the small functions should be inline, and what inline actually asks the compiler to do

• What would be lost if the private members were made public instead

Correctness Check

Test with times on either side of noon and the same time twice, and verify by

hand. Growth Investigation

Measure the time for 10,000,000 calls with and without inline (compile with -O0 and with - O2), and explain the result.

Deployment Decision

Decide whether the friend functions should stay or be replaced by member functions and getters, and defend your decision.

Complete the Code

Complete TODO 1 and TODO 2 below inside the friend functions, and execute the

program. #include <iostream>

class Time {

```
 int h, m;

public:
 Time(int hh, int mm) : h(hh), m(mm) {}
 friend int minutesBetween(const Time& a, const Time& b);
 friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
 // TODO 1: Using the private members h and m, return the absolute difference between
// the two times in minutes

}

bool earlier(const Time& a, const Time& b)
{
 // TODO 2: Return true if a is earlier in the day than b

}

int main()
{
 Time departure(9, 45), arrival(13, 10);
 std::cout << minutesBetween(departure, arrival) << ' '
 << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

205 false

Test the same time twice and a pair of times on either side of noon.

Alternative Concept and Complexity Analysis

If the comparison were written as member functions (a.minutesTo(b), a.isEarlier(b)) or with public getters instead of friend functions, what would the solution be? Analyse the time per call (the same after inlining, O(1)) and measure it at -O0 and -O2, and compare the encapsulation of the three designs.

Evidence Expected

A justified friend-function design, correctness tests, measured inline timings at two optimisation levels and an evidence-based decision.

DAY-4 DESIGN CHALLENGE

ANSWER:

Friend Functions and Inline Functions

1. Completed C++ Program

TODO 1: Convert both times into minutes and calculate the absolute difference.

TODO 2: Compare the total minutes of both objects.

```
#include <iostream>

#include <cstdlib>

using namespace std;

class Time {
```

```
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
 friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
    int t1 = a.h * 60 + a.m;
    int t2 = b.h * 60 + b.m;
   return abs(t1 - t2);
}
bool earlier(const Time& a, const Time& b)
{
    int t1 = a.h * 60 + a.m;
```

```
    int t2 = b.h * 60 + b.m;



    return t1 < t2;

}

int main()

{

    Time departure(9, 45), arrival(13, 10);

 cout << minutesBetween(departure, arrival) << ' '

         << boolalpha << earlier(arrival, departure) << '\n';

 Time t1(11, 50), t2(12, 10);

    cout << minutesBetween(t1, t2) << ' '

         << earlier(t1, t2) << '\n';

Time t3(10, 30), t4(10, 30);

    cout << minutesBetween(t3, t4) << ' '

         << earlier(t3, t4) << '\n';
```

```
 return 0;

}
```

2. Output

205 false

20 true

0 false

Verification:

- 9:45 to 13:10 = 205 minutes.
- 11:50 to 12:10 = 20 minutes (across noon).
- 10:30 to 10:30 = 0 minutes.
- Same times are not earlier, so the result is false.

3. Why is a Friend Function Needed?

A friend function is used because it needs to access the private data members of the Time class.

- It can access private members h and m.
- It can compare two objects equally.

- It is declared inside the class using the friend keyword.

- It is defined outside the class.

Reason: Since the function compares two objects, a non-member friend function is a suitable design.

4. Friend Function vs Member Function

Friend function	Member function
Defined outside the class	Defined inside the class
Called directly	Called using an object
Can access private members	Can access private members
Uses two objects as arguments	Usually uses one object and another as argument

Example:

// Friend function

minutesBetween(a, b);

// Member function

a.minutesTo(b);

5. Why Use Inline Functions?

Inline functions are suitable for small functions that are called frequently.

- They request the compiler to replace a function call with the function's code.
- This can reduce function-call overhead.
- They are useful for small calculations.
- The compiler may ignore the inline request.

Important: Inline does not guarantee that the compiler will insert the function code directly.

6. Growth Investigation – Performance

We can measure 10 million calls using chrono.

Example compilation commands:

g++ program.cpp -O0 -o test

./test

g++ program.cpp -O2 -o test

./test

- -O0: No compiler optimisation.
- -O2: Higher compiler optimisation.

- At -O0, inline functions may still behave like normal function calls.

- At -O2, the compiler may automatically inline small functions.

The execution time depends on the processor, compiler and system load. Therefore, repeated measurements are recommended.

Time complexity:

Operation	Time complexity
minutesBetween()	O(1)
earlier()	O(1)
10 million calls	O(n)

Both functions perform a fixed number of calculations, so each call takes constant time.

7. What If Private Members Are Made Public?

If h and m are made public:

- Any function can directly modify the time values.

- Data protection is reduced.

- Invalid values can be stored.

- Encapsulation is lost.

Therefore, private members should remain private.

8. Alternative Designs and Complexity Analysis

Design A – Friend functions

minutesBetween(a, b);

earlier(a, b);

Advantages:

- Direct access to private data.
- Suitable for comparing two objects equally.
- No need for public getters.

Design B – Member functions

a.minutesTo(b);

a.isEarlier(b);

Advantages:

- Better object-oriented structure.
- No need for friend access.

- Easy to understand and maintain.

Design C – Public getters

a.getHour();

a.getMinute();

Advantages:

- Private data remains private.

- Other functions can access values through getters.

Disadvantage:

- More function calls and unnecessary exposure of internal data.

9. Final Deployment Decision

I would retain the friend functions in this design because both functions compare two Time objects equally. They need direct access to private members without making them public.

Inline can be used for these small functions, but the compiler decides whether to actually inline them.

Member functions are also a good alternative if the operations are considered responsibilities of a single Time object.

Conclusion: Friend functions provide a good balance between encapsulation, simplicity and performance. Both friend and member designs have O(1) time complexity per call.

Q7 – Diagnose, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE program that shows a significant limitation as the data grows or the requirement changes.

You are not permitted simply to replace your code with a library facility and claim it is better. You must understand the problem first and then develop an appropriate redesign.

A. Diagnose the Limitation – 2 Marks

Using your own evidence, identify where the design fails or slows down and which characteristic of the original design causes it.

Evidence Expected

A precise diagnosis supported by evidence from your original

program.

B. Define the Design Objective – 2 Marks

State a measurable objective for the redesign.

Evidence Expected

A clearly stated objective derived from the identified limitation.

C. Design the Revised Solution – 4 Marks

Explain what you have changed in the class or function design, which C++ feature you use, whether extra memory is needed, and any new trade-off. Provide a class diagram or pseudocode.

Evidence Expected

A coherent redesign directly addressing the diagnosed limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate the Redesign – 2 Marks

Compare Original → Redesigned using the same tests and

measurements. Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Evaluate whether the objective was achieved, new costs, and where the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible conclusion.

Submission Requirements
GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Q1–Q7 •

C++ source code for all programs

• Predictions, outputs, timings and compiler messages used as evidence

ANSWER:

Diagnose, Redesign and Validate

For this question, we need to select one program from Q1–Q6, identify its limitation, redesign it and compare the original and redesigned versions.

I will use the Train Journey Planner (Friend Functions and Inline Functions) as the example.

A. Diagnose the Limitation

The original program uses two friend functions, minutesBetween() and earlier(), to compare two Time objects.

Limitations:

1. Both functions separately calculate total minutes.
2. Repeated calculations can increase unnecessary work.
3. When more time-related operations are added, code duplication may increase.
4. The original design does not validate invalid hour and minute values.

Evidence from the original program:

int t1 = a.h * 60 + a.m;

int t2 = b.h * 60 + b.m;

The same calculation is repeated in both friend functions.

B. Define the Design Objective

The objective is to redesign the Time class to avoid repeated calculations and improve code maintainability.

Measurable objectives:

- Calculate total minutes only once per object.
- Reduce duplicate calculation statements.
- Validate hours (0–23) and minutes (0–59).
- Maintain O(1) time complexity for each comparison.
- Preserve private data members.

C. Design the Revised Solution

Changes made:

1. Use a private helper function called toMinutes().
2. Use member functions for comparison.
3. Keep hours and minutes private.
4. Add input validation using the constructor.
5. Avoid unnecessary friend-function access.

C++ features used:

- Classes and objects
- Private members
- Member functions
- const functions
- Inline function (optional)

Memory requirement: Only a small amount of additional memory may be used for temporary calculations. No additional data structure is required.

Revised Class Diagram

Time Class

- h : int

- m : int

- total : int

+ Time(hour, minute)

+ minutesBetween(other)

+ isEarlier(other)

+ toMinutes()

Revised C++ Program

```
#include <iostream>
#include <stdexcept>
#include <cstdlib>
using namespace std;

class Time {
private:
   int h, m;
   int total;

   int toMinutes() const {
      return h * 60 + m;
   }

public:
   Time(int hh, int mm) {
      if (hh < 0 || hh > 23 || mm < 0 || mm > 59)
         throw invalid_argument("Invalid time");

      h = hh;
      m = mm;
      total = toMinutes();
   }
```

```
    int minutesBetween(const Time& other) const {
        return abs(total - other.total);
    }

    bool isEarlier(const Time& other) const {
        return total < other.total;
    }
};

int main() {
    try {
        Time departure(9, 45);
        Time arrival(13, 10);

        cout << departure.minutesBetween(arrival) << " ";
        cout << boolalpha
             << arrival.isEarlier(departure) << endl;

        Time t1(11, 50), t2(12, 10);
        cout << t1.minutesBetween(t2) << " "
             << t1.isEarlier(t2) << endl;

        Time t3(10, 30), t4(10, 30);
        cout << t3.minutesBetween(t4) << " "
             << t3.isEarlier(t4) << endl;
    }
    catch (const invalid_argument& e) {
        cout << e.what() << endl;
    }
```

```
    return 0;
}
```

Revised output:

205 false

20 true

0 false

The redesigned program produces the same correct results as the original program.

D. Predict Before Implementing

Before implementation, I predict that:

- The redesigned program will perform fewer calculations during repeated comparisons.
- It will avoid repeatedly converting hours and minutes into total minutes.
- It will improve performance when the same Time objects are compared millions of times.
- Each comparison will still have O(1) time complexity.

The expected improvement will be measured using execution time in milliseconds.

E. Implement and Validate the Redesign

Both programs must be tested using the same input values and compiler settings.

Comparison of results:

Test case	**Original**	**Redesigned**
9:45 and 13:10	205	205
11:50 and 12:10	20	20
10:30 and 10:30	0	0
Invalid time 25:70	Not validated	Rejected

Performance testing:

Compile both versions using:

g++ original.cpp -O0 -o original

g++ redesigned.cpp -O0 -o redesigned

g++ original.cpp -O2 -o original2

g++ redesigned.cpp -O2 -o redesigned2

Run each program 10,000,000 times using the same test data.

Record the actual execution time:

Compiler option	Original time	Redesigned time
-O0	Record ms	Record ms
-O2	Record ms	Record ms

Note: These timing values must be measured on your computer. Do not enter assumed values as experimental results.

Expected observation:

- At -O0, the redesigned version may show a performance improvement because it avoids repeated conversions.
- At -O2, the compiler may optimise both versions, reducing the difference.
- Both versions have O(1) complexity per comparison.

F. Defend the Final Design

Advantages of redesigned solution:

1. Avoids repeated time conversion.
2. Improves code reusability and maintainability.
3. Validates incorrect time values.
4. Maintains data security using private members.
5. Provides faster repeated comparisons when the compiler does not eliminate the original repeated calculations.

New costs:

- Requires extra memory to store total.
- Constructor becomes slightly more complex.
- The cached value must be updated if time modification is introduced.

When is the original design preferable?

The original friend-function design is preferable for small programs where comparisons are performed only a few times. It uses less stored data and is simple to understand.

Final conclusion:

The redesigned Time class is better for repeated journey comparisons because it calculates

total minutes once during object creation and reuses the result. It reduces repeated calculations while maintaining O(1) time complexity. Therefore, the redesign meets the main objective, although actual performance improvement must be confirmed through benchmarking.

Assessment Verification

Students may be asked to explain any part of their predictions, code, tests or results during a brief verification discussion/viva.